

Optimisation de la réduction d'expressions du lambda-calcul

HUROT Eliott (30348) — MPI, 2025–2026

Table des matières

1. Fondements théoriques	2
1.1. Syntaxe et réductions	2
1.2. Terminaison et confluence	2
1.3. Stratégies de réduction et standardisation	3
2. Implémentation de l'interpréteur	3
2.1. Algorithme	4
3. Recherche du chemin optimal par A*	4
3.1. Formulation comme problème de plus court chemin	4
3.2. Algorithme A*	4
3.3. Heuristiques admissibles pour le lambda-calcul	5
3.4. Optimisations notables	5
3.5. Structures de données pour la file de priorité	6
4. Conclusion	6
5. Annexe	7
Bibliographie	8

Introduction

Le lambda-calcul, introduit par Alonzo Church dans les années 1930, formalise la notion de calcul par application et substitution de variables. Il constitue le fondement théorique des langages fonctionnels modernes tels que Haskell ou OCaml [1], [2], dans lesquels l'exécution d'un programme correspond précisément à la *béta-réduction* d'un lambda-terme.

Une propriété fondamentale de ce modèle est que la forme normale d'un terme — son résultat de calcul — est unique lorsqu'elle existe (1). En revanche, les *chemins* pour l'atteindre sont multiples, et leur longueur peut varier considérablement.

L'objectif est donc :

- implémenter un interpréteur supportant diverses stratégies;
- construire le graphe de réductions de manière paresseuse;
- déterminer la réduction minimale par A*.

1. Fondements théoriques

1.1. Syntaxe et réductions

Définition 1 (Lambda Terme)

L'ensemble Λ des **lambda-termes** est défini inductivement par :

$$\Lambda := x \quad | \quad MN \quad | \quad \lambda x.M$$

où x parcourt un ensemble dénombrable de variables V , MN est l'*application* de M à N , et $\lambda x.M$ est l'*abstraction* de x dans M . Un sous-terme de la forme $(\lambda x.u), v$ est appelé un **redex**.

L'**alpha-équivalence** $=_\alpha$ identifie les termes ne différant que par le renommage cohérent de variables liées : $\lambda x.x =_\alpha \lambda y.y$. On travaille dans le quotient $\Lambda/=_\alpha$, implémenté par une comparaison structurelle.

Définition 2 (Béta-réduction)

La **béta-réduction** $\rightarrow_\beta$ est la plus petite relation sur $\Lambda/=_\alpha$ vérifiant :

$$(\lambda x.u)v \rightarrow_\beta u[x := v]$$

et stable par contexte (application et abstraction). Sa clôture réflexive-transitive est notée $\rightarrow_\beta^*$. Un terme sans redex est appelé **forme normale**.

La **béta-réduction ne termine pas en général** : le terme $\Omega := (\lambda x.xx)(\lambda x.xx)$ satisfait $\Omega \rightarrow_\beta \Omega$ et n'a pas de forme normale.

1.2. Terminaison et confluence

On distingue deux niveaux de normalisation :

- **Fortement normalisant (SN)** : toutes les suites de réductions de u sont finies.
- **Faiblement normalisant (WN)** : il existe au moins une suite finie menant à une forme normale.

Théorème 1 (Confluence)

Si $u \rightarrow_\beta^* v_1$ et $u \rightarrow_\beta^* v_2$, alors il existe w tel que $v_1 \rightarrow_\beta^* w$ et $v_2 \rightarrow_\beta^* w$.

Démonstration. On introduit la **réduction parallèle** $\rightarrow_\parallel$, qui contracte simultanément un ensemble de redex disjoints. Elle satisfait $\rightarrow_\beta \subseteq \rightarrow_\parallel \subseteq \rightarrow_\beta^*$. L'étape clé est la **propriété du diamant pour $\rightarrow_\parallel$** : si $u \rightarrow_\parallel v_1$ et $u \rightarrow_\parallel v_2$, il existe w tel que $v_1 \rightarrow_\parallel w$ et $v_2 \rightarrow_\parallel w$. Ce w s'obtient en contractant dans v_1 (resp. v_2) les redex créés par la réduction vers v_2 (resp. v_1) ; la compatibilité des substitutions parallèles assure la cohérence. La confluence de $\rightarrow_\beta^*$ s'obtient alors par induction sur la longueur des séquences [1, ch. 3]. $\square$

Corollaire 1

La forme normale d'un terme, lorsqu'elle existe, est **unique à $=_\alpha$ près**. Ce corollaire justifie la notion de chemin *optimal* : tous les chemins mènent au même but unique.

1.3. Stratégies de réduction et standardisation

Une **stratégie** est une fonction sélectionnant le prochain redex à contracter. Les deux stratégies canoniques sont :

- **Stratégie interne** (appel par valeur) : on évalue l'argument avant substitution. Correspond aux langages stricts (C, OCaml par défaut).
- **Stratégie externe gauche** (ordre normal) : on réduit le redex le plus à gauche le plus à l'extérieur. C'est la seule stratégie garantie de terminer lorsqu'une forme normale existe.

Ces stratégies ne sont pas équivalentes : $(\lambda x.y), \Omega \rightarrow_\beta y$ en stratégie externe (1 pas), mais diverge en stratégie interne.

Théorème 2 (Standardisation)

Si u est faiblement normalisant, la stratégie externe gauche calcule la forme normale de u en un nombre fini de pas.

Démonstration. Une **réduction standard** est une séquence où aucun redex n'est contracté à gauche d'un redex précédemment contracté. Le théorème de standardisation (Curry, 1958) affirme que toute réduction peut être réordonnée en réduction standard [2]. La stratégie externe gauche produit précisément de telles séquences. Si le terme est WN, la réduction standard doit atteindre la forme normale : une réduction standard bloquée sur un terme non-normal contiendrait un redex contractable à droite, contradiction. La terminaison découle d'un ordre bien fondé sur la position du redex contracté à chaque étape. $\square$

Conséquence. Le théorème de standardisation garantit que notre programme termine pour tout terme WN, même si A^* ne trouve pas le chemin optimal dans le temps imparti.

2. Implémentation de l'interpréteur

La gestion de l'alpha-équivalence est cruciale : sans elle, des termes identiques seraient traités comme distincts dans le graphe de réductions, causant des explosions combinatoires. La comparaison structurelle des termes permet de pallier ce problème efficacement.

2.1. Algorithme

Le programme transforme une chaîne de caractères en chemin optimal par quatre étapes :

Étape	Transformation	Note de complexité
1	Chaîne $\rightarrow$ Tokens	$O(n)$
2	Tokens $\rightarrow$ AST	$O(n)$
3	AST $\rightarrow$ Graphe	$O(n^3)$ par sommet
4	Graphe $\rightarrow$ Plus court chemin	$O(M \log(N))$

La construction du graphe est **paresseuse** : les successeurs d'un nœud ne sont calculés que lors de son extraction de la file de priorité. Cette approche est indispensable car le graphe peut être infini (terme divergent) ou d'une taille rédhibitoire ($[P][10]$: 50 000 sommets et 310 000 arêtes).

Les complexités des opérations élémentaires (avec n = nombre de nœuds, m = taille du terme substitué) sont :

Opération	Complexité
Égalité structurelle	$O(n \cdot v)$
Substitution de x par v dans u	$O(n + m^2)$
Étape de réduction complète	$O(n^3)$

3. Recherche du chemin optimal par A*

3.1. Formulation comme problème de plus court chemin

Le **graphe de réductions** G_u d'un terme u est le graphe orienté dont les sommets sont les termes accessibles depuis u par $\rightarrow_\beta^*$, et les arêtes sont les béta-réductions élémentaires. La forme normale de u , si elle existe, est l'unique puits de ce graphe (par confluence). Le problème est donc : trouver le chemin de poids minimal de u vers sa forme normale dans G_u .

3.2. Algorithme A*

Théorème 3 (Optimalité de A*)

L'algorithme A* retourne un plus court chemin de la source vers le but si et seulement si l'heuristique h est **admissible** : $\forall u, h(u) \leq d^*(u)$, où $d^*(u)$ est le nombre minimum de réductions pour normaliser u .

A* sélectionne à chaque étape le nœud minimisant $f(u) = g(u) + h(u)$, où $g(u)$ est le nombre de réductions déjà effectuées depuis le terme initial.

L'exemple $(\lambda x.((\lambda x.[I])\Omega)([+][xx])([+][1][2]))$ illustre la non optimalité des stratégies de réduction basiques :

Méthode	Taille de la réduction
Stratégie interne	$+\infty$
Stratégie externe gauche	21
A* (heuristique h_2)	15

3.3. Heuristiques admissibles pour le lambda-calcul

Plusieurs heuristiques ont été testées mais 2 seulement se sont révélées admissibles et celle retenue est fondée sur la notion de spine redex [3].

$h(u)$ = Nombre de spine redex de u

Définition 3 (Redex Nécessaire)

Un redex R dans un terme M est dit **nécessaire** si, dans **toute** réduction de M menant à sa forme normale, un résidu de R est contracté.

Conséquence pour A* : Le nombre de ces redex nécessaires constitue une borne inférieure de la distance minimale de u à sa forme normale. Dès lors, l'heuristique h est admissible.

Théorème 4 (Indécidabilité)

Déterminer si un redex quelconque est nécessaire est indécidable.

On ne peut donc pas déterminer leur nombre directement mais les algorithmes et la notion de spine redex développés par Barendregt permet d'obtenir une borne inférieure de ce nombre de redex nécessaire [3].

3.4. Optimisations notables

L'application de l'algorithme A* au lambda calcul a permis d'obtenir deux optimisations spécifiques au lambda-calcul qui, bien que dépendantes des termes, peuvent se révéler bénéfiques :

- L'utilisation d'une comparaison secondaire lors d'une égalité dans la file de priorité permet d'orienter A* en préservant sa correction;
- Lorsqu'un terme est en forme normale de tête $\lambda x_1, \dots, x_n. x u_1 \dots u_m$ les réductions des u_i sont indépendantes, dès lors, réduire u_i avant ou après u_j ne change pas la taille de la réduction minimale. Ainsi en forçant la réduction de ces termes de gauche à droite, on retire dans le graphe tous les chemins constituant des permutations des réductions des u_i sans briser la correction de A*.

Optimisation	$\langle [+][2][2], [*][2][2] \rangle$		$(\lambda x.(\lambda y.[I])\Omega([+][xx]))([+][1][2])$		$[P][8]$	
	Temps	$ V $	Temps	$ V $	Temps	$ V $
Aucune	0,138s	182	11s	4 568	29s	7 377
Comparaison bis	0,137s	181	10s	4 521	29s	7 375
Ordre de réduction	0,119s	26	10s	4 568	28s	7 377
Les deux	0,116s	26	10s	4521	29s	7 375

3.5. Structures de données pour la file de priorité

L'efficacité de A^* repose sur celle de la file de priorité (extraction du minimum et insertion).

Les complexités théoriques divergent : le tas de Fibonacci offre $O(1)$ amorti en insertion et $O(\log n)$ en extraction, contre $O(\log n)$ pour le tas binaire dans les deux cas [4]. L'avantage pratique dépend toutefois de la localité mémoire et des constantes cachées, d'où la nécessité de tests réels :

Structure	$\langle [+][2][2], [*][2][2] \rangle$	$(\lambda x.(\lambda y.[I])\Omega([+][xx]))([+][1][2])$	$[P][8]$
Tas binaire	0,0013s	1m30s	6m48s
Tas binomial	0,0012s	16s	1m31s
Tas de Fibonacci	0,0014s	11s	30s

4. Conclusion

Ce TIPE a formalisé la recherche de stratégie de réduction optimale comme un problème de plus court chemin dans un graphe potentiellement infini, et a conduit au développement d'un interpréteur OCaml complet mettant en œuvre A^* avec des heuristiques spécifiques au lambda-calcul. Les résultats confirment un gain significatif (15 pas contre 21 sur l'exemple présenté). L'optimisation de l'algorithme ouvre des perspectives d'amélioration par le choix de la file de priorité et l'affinement des heuristiques afin d'être en capacité de traiter des termes plus conséquent.

Finalement, malgré des optimisations permettant de réduire la taille des graphes explorés et les complexités des opérations, de nombreux termes restent hors d'atteinte dans des limites de temps et de mémoire raisonnables. Aussi il pourrait alors être intéressant de s'intéresser aux approximations de ces réductions minimales pour augmenter la portée de notre outil en perdant une certaine part d'information.

5. Annexe

Graphes illustrant l'optimisation sur l'ordre des dérivations d'un terme en forme normale de tête appliquée à $\langle [+][2][2], [*][2][2] \rangle$

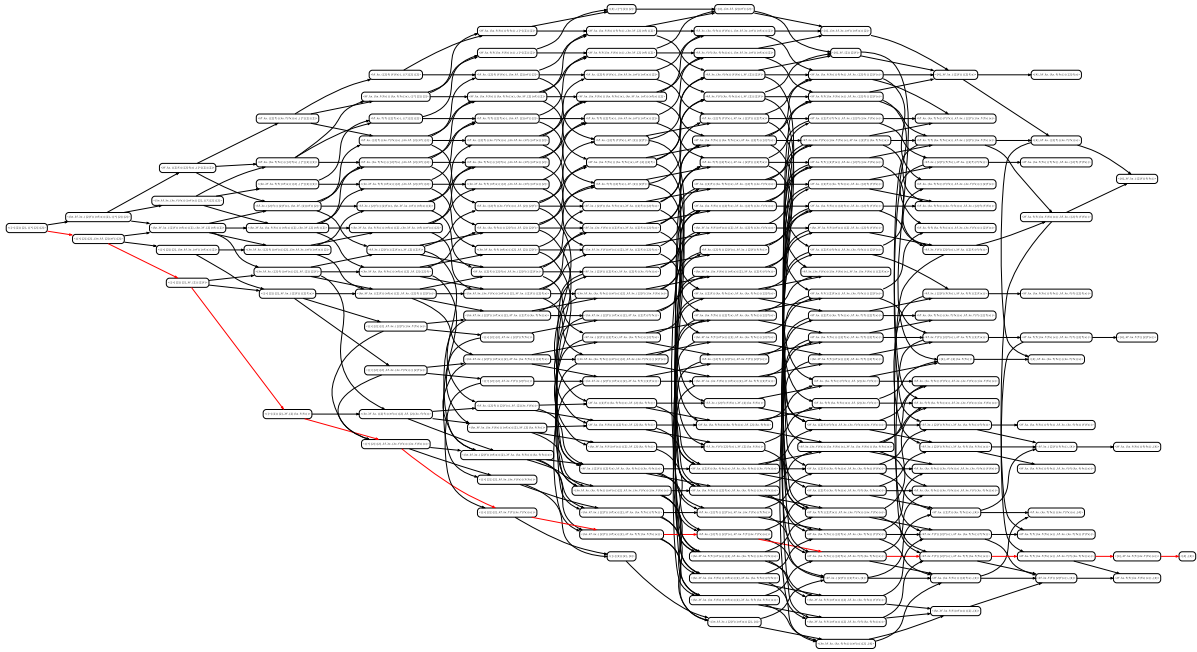


Fig. 1. – Sans optimisation

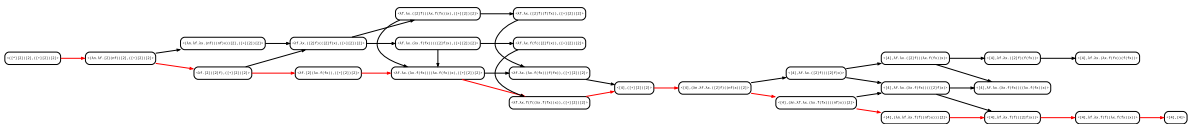


Fig. 2. – Avec optimisation

Bibliographie

- [1] H.P. Barendregt, *The lambda Calculus - Its Syntax and Semantics*.
- [2] Jean Goubault-Larrecq, « Lambda-calcul et langages fonctionnels ». [En ligne]. Disponible sur: <https://lsv.ens-paris-saclay.fr/~goubault/Lambda/lambda.pdf>
- [3] H.P. Barendregt, « Needed Reduction and Spine Strategies for the Lambda Calculus ». [En ligne]. Disponible sur: <https://www.sciencedirect.com/science/article/pii/0890540187900010?via%3Dihub>
- [4] « D-ary heap - Wikipedia ». [En ligne]. Disponible sur: https://en.wikipedia.org/wiki/D-ary_heap